Informe de entrega — TP Integrador de Calidad de Software aplicada al TFG

Proyecto: Canchero — plataforma de reserva de canchas deportivas **Repositorio:** https://github.com/felipeganame/CANCHERO-V2 **Pipeline:** https://github.com/felipeganame/CANCHERO-V2/actions **SonarCloud:** https://sonarcloud.io/project/overview?id=felipeganame_CANCHERO-V2 **Fecha de este informe:** 2026-10-24 **Nivel declarado:** Nivel 3 — prototipo avanzado

Este documento es el punto de entrada para leer la entrega. Resume qué se construyó, cómo se verificó, y en qué orden conviene recorrer el resto de `docs/` para la defensa oral. Cada afirmación de este informe tiene su evidencia detallada en un documento aparte — acá no se repiten números, se referencian.

1. Resumen ejecutivo

Canchero es una app web donde un **jugador** busca canchas disponibles por deporte/ciudad/fecha, reserva un turno y lo cancela si hace falta; y un **club** administra sus canchas (crearlas, deshabilitarlas, ver sus reservas) desde un panel propio. Dos roles, dos flujos, una sola base de reglas de negocio (RN-01 a RN-10, ver `docs/trazabilidad.md`).

Estado actual: las 10 reglas de negocio están implementadas y probadas en 4 niveles (unitario, integración, API, E2E); el login y registro son reales (no un "login de prueba"): alta con verificación de email por token (Resend), inicio de sesión con email/contraseña, y recuperación de contraseña, todo vía Better Auth. El pipeline de CI/CD tiene 4 etapas (calidad → deploy QA → verificación QA → deploy producción) con un quality gate de SonarCloud que bloquea de verdad, no de forma cosmética. Hay 2 entornos desplegados con URLs estables:

- **QA:** https://canchero-qa.vercel.app
- **Producción:** https://canchero-puce.vercel.app

Qué quedó explícitamente fuera de alcance, por decisión, no por olvido: login con Google (el botón y el `socialProvider` están comentados en el código, ver sección 6) y el mapa de resultados en `/buscar` (también comentado, pendiente de integrar Leaflet). Ambos están documentados como decisión de alcance en el propio código, con un comentario explicando por qué.

2. Arquitectura

Capa	Tecnología
Framework	Next.js 15 (App Router), React 19, TypeScript estricto
Estilos	Tailwind CSS, tokens de diseño propios (bordo/naranja/ink/muted/...)
Base de datos	PostgreSQL gestionado por Supabase, acceso vía Drizzle ORM + `postgres-js`
Autenticación	Better Auth: email/contraseña real, verificación de cuenta por token (Resend), recuperación de contraseña (Resend). Google OAuth implementado pero comentado (sección 6)
Email transaccional	Resend: verificación de cuenta, recuperación de contraseña, confirmación/cancelación de reserva
Validación	Zod en los límites de API (`src/lib/validation/`)

Testing	Vitest (unit/componentes/API/integración), Testcontainers (Postgres real), Playwright (E2E/smoke), k6 (stress)
CI/CD	GitHub Actions, 4 etapas, deploy a Vercel
Análisis estático	SonarCloud, quality gate bloqueante

Patrón central: `domain/` es lógica pura sin I/O; `server/services/` son casos de uso que reciben repositorios inyectados por interfaz (`server/repositories/types.ts`), con una implementación real (Drizzle) y una falsa (doble de prueba, `server/repositories/fakes.ts`). Esto es lo que permite que 154 de los 162 tests (unitarios, de componentes y de API) corran con dobles de prueba, sin tocar una base de datos real, mientras que los 2 tests de integración por Testcontainers (RN-01, RN-05) corren contra Postgres real genérico, los 2 tests de integración equivalentes corren contra el proyecto Supabase de QA real (etapa `deploy-qa`), y los 4 de E2E/smoke corren contra el entorno de QA realmente desplegado. Ver `docs/integracion-externa.md` para el detalle completo.

3. Cómo correr el proyecto

Local

```
pnpm install
pnpm db:migrate        # aplica el schema a Postgres local
pnpm dev               # localhost:3000
```

Variables de entorno en `.env.local` (ver `.env.example`): `DATABASE_URL` , `BETTER_AUTH_SECRET` , `BETTER_AUTH_URL=http://localhost:3000` , `RESEND_API_KEY` , `RESEND_FROM` .

Si `pnpm dev` se queda colgado compilando alguna página (pasó de forma intermitente durante el desarrollo, sin causa raíz identificada), usar `pnpm build && pnpm start` en su lugar — es el modo que se usó para todo el testeo manual de este proyecto.

Verificación completa antes de un commit

```
pnpm typecheck && pnpm lint && pnpm test:unit && pnpm build
```

Entornos desplegados

El pipeline (`.github/workflows/ci.yml`) despliega con `vercel deploy` y fija un alias estable para que QA y producción tengan URLs legibles y nunca cambien, aunque Vercel genere una URL nueva en cada deploy:

```
# etapa deploy-qa
url=$(npx vercel deploy --token=...)
npx vercel alias set "$url" canchero-qa.vercel.app --token=...

# etapa deploy-prod
npx vercel deploy --prod --token=...   # alias estable de producción ya configurado en Vercel
```

Entorno	URL	Cuándo se actualiza
QA	https://canchero-qa.vercel.app	En cada push a `main`, antes de producción
Producción	https://canchero-puce.vercel.app	Solo si QA pasó smoke + E2E + stress

4. Mapa de la documentación (orden sugerido para la defensa)

1. **`docs/objetivos-calidad.md`** — qué se propuso el equipo lograr (usuario, proceso, organización) y el plan de contingencia si cae una dependencia crítica. Punto de partida natural: todo lo demás es evidencia de que esto se cumplió o no.
2. **`docs/riesgos.md`** — riesgos identificados con probabilidad × impacto, incluyendo los que ya se materializaron durante el desarrollo (y cómo se resolvieron).
3. **`docs/trazabilidad.md`** — la matriz RN → CP → defecto. Es el documento más denso en evidencia concreta: cada regla de negocio apunta a los tests exactos que la prueban, en qué nivel.
4. **`docs/plan-de-pruebas.md`** — alcance, niveles de prueba, entornos, criterios de entrada/salida por etapa del pipeline, técnicas de diseño de casos (clases de equivalencia, límites, tablas de decisión).
5. **`docs/metricas.md`** — SonarCloud antes/después (cobertura 21% → 87% local / 83% nuevo código en Sonar), y el hallazgo real de stress (límite de concurrencia del free tier de Vercel).
6. **`docs/defectos.md`** — los 4 defectos reales encontrados durante el desarrollo normal, con pasos, causa raíz y corrección. No son defectos fabricados para la entrega.
7. **`docs/iso25010.md`** — las 8 características de calidad del producto con evidencia concreta por cada una, y el top 3 justificado.
8. **`docs/qa-vs-qc.md`** — qué prácticas de este proyecto son QA (preventivas) y cuáles son QC (verificación), y por qué la distinción importa para este TP específicamente.
9. **`docs/integracion-externa.md`** — Supabase como integración principal (contrato, pruebas contra el doble y contra el entorno real), más Vercel y Resend como integraciones secundarias.
10. **`docs/herramienta-nueva.md`** — proceso de 5 pasos que justifica la incorporación de k6 al proyecto.

5. Cómo defender cada sección (lo más fuerte de cada una)

- **Objetivos de calidad:** no son aspiracionales sueltos, cada uno tiene una columna "cumplido" con evidencia verificable (número de SonarCloud, ID de test, corrida real del pipeline).
- **Riesgos:** el R2 (agotamiento del pool de Postgres) y el R3 (límite de concurrencia de Vercel) no son riesgos teóricos de manual — se materializaron de verdad en este desarrollo y están documentados como tal en `docs/defectos.md` y `docs/metricas.md`.
- **Trazabilidad:** el punto fuerte es que cada defecto real lo encontró una capa distinta de la pirámide de pruebas (integración, E2E, análisis estático) — es el argumento de que una sola capa no alcanza, con prueba, no con teoría.
- **Plan de pruebas:** los criterios de salida por etapa son los que realmente bloquean el pipeline (`ci.yml`), no una lista aspiracional separada del código.
- **Métricas:** la cobertura subió de 21% a 87% agregando tests con el mismo patrón de doble de prueba que ya existía desde el día 1 — evidencia de que la arquitectura en capas facilita testear, no que "se escribieron muchos tests porque sí".
- **Defectos:** DEF-01 (concurrencia) y DEF-04 (pool de conexiones) son los más defendibles: ambos se encontraron bajo condiciones reales (Postgres real, carga concurrente real contra infraestructura compartida), no con datos sintéticos fáciles de armar a propósito.
- **ISO 25010:** el top 3 (fiabilidad, seguridad, mantenibilidad) está justificado con evidencia dura, no autoevaluación — por ejemplo, RN-01 no depende solo del código: hay un índice único en la base

que lo garantiza aunque el código falle.
- **QA vs QC:** el argumento central es que encontrar 4 defectos reales durante desarrollo normal (no fabricados para la entrega) es exactamente lo que debería producir QA+QC combinados funcionando de verdad, no una checklist.
- **Integración externa:** DEF-04 es la joya de esta sección — un defecto que es literalmente imposible de encontrar sin probar contra el proveedor real (Supabase), porque depende de un límite de conexiones que no existe en Postgres local.
- **Herramienta nueva (k6):** la decisión no fue "porque sí es popular": la matriz de decisión pesa la integración nativa a GitHub Actions y que el equipo ya sabe TypeScript, evitando sumar un runtime nuevo (descartando Locust/Python) por esa razón explícita.

6. Autenticación real: qué se construyó y qué quedó fuera a propósito

Hasta hace pocos commits (`1d44db0`), el login usaba un atajo de prueba. Se reemplazó por completo:

- **Registro:** `signUp.email` crea la cuenta y dispara un email de verificación con token (Resend); la cuenta no puede iniciar sesión hasta confirmar el link (`requireEmailVerification: true` en `src/integrations/auth/index.ts`).
- **Login:** `signIn.email` real contra Better Auth; si la cuenta no verificó el email, la API devuelve 403 y la UI lo explica claramente ("Todavía no confirmaste tu cuenta").
- **Recuperación de contraseña:** `requestPasswordReset` + `resetPassword` , también vía Resend, con el mismo mensaje de éxito exista o no la cuenta (anti-enumeración de usuarios).
- **Cuentas de seed (E2E/CI):** no pueden abrir un email real, así que se marcan `emailVerified: true` directamente al crearlas (`src/server/db/seed-e2e.ts` , `src/server/db/seed.ts`).

Decisiones de alcance documentadas en el propio código, no ocultas:

- **Google OAuth:** el proveedor y el botón están implementados pero comentados (`src/integrations/auth/index.ts` línea 20, `src/app/login/login-form.tsx` línea 8) hasta tener credenciales reales de Google Cloud Console. No es un bug, es una decisión de alcance explícita con fecha de cuándo se retomaría.
- **Mapa de resultados (`/buscar`):** el bloque está comentado (`src/app/buscar/buscar-results.tsx` línea 262) hasta integrar Leaflet + OpenStreetMap. Misma lógica: decisión de alcance, no omisión.

Nota sobre `docs/riesgos.md` (R1) y `docs/plan-de-pruebas.md` : ambos documentos originalmente listaban "falta login real de Google" como el riesgo/pendiente más alto, de cuando el login todavía era un atajo de prueba. Ya se actualizaron los dos: R1 pasó de prioridad 9 ("bloqueante") a prioridad 1 ("resuelto por alcance, Google queda comentado a propósito"), y el plan de pruebas ya no dice "pendiente de login real" — el login real está implementado y probado; lo único pendiente es específicamente Google OAuth como método adicional.

7. Limitaciones conocidas (honestas, no ocultas)

- **Envío de email a destinatarios arbitrarios:** resuelto durante esta entrega. La capa gratuita de Resend solo permite enviar a la casilla verificada de la cuenta hasta verificar un dominio propio (registros DNS) — se verificó `canchero.ganamewebstudio.com` como subdominio de envío (DKIM/SPF/DMARC vía Hostinger), y `RESEND_FROM` se actualizó en Vercel (preview y producción) y en local. Confirmado con envíos reales de prueba (registro + recuperación de contraseña) a una casilla externa, con status 200 en los logs de Resend tanto en QA como en producción.
- **Accesibilidad automatizada (Axe/Lighthouse):** no está integrada al pipeline todavía (ver `docs/iso25010.md` , característica de Usabilidad).

- **Cross-browser:** las pruebas E2E corren solo contra Chromium vía Playwright.
- **Limpieza de datos de prueba en QA:** las canchas `E2E Temp...` creadas por tests repetidos se acumulan en la base de QA (riesgo R5 en `docs/riesgos.md`); no hay job de limpieza automática todavía.

Ninguna de estas limitaciones es un defecto escondido: cada una está documentada en el lugar correspondiente de `docs/`, con el motivo y, cuando aplica, el siguiente paso concreto.

8. Nivel declarado y cumplimiento de metas

Nivel declarado: Nivel 3 — prototipo avanzado.

8.1 Metas de calidad (tabla del enunciado)

Aspecto	Exigido (Nivel 3)	Estado	Evidencia
Pipeline	Compilación, pruebas, análisis estático, cobertura, quality gate, reporte automático	**Cumplido**	`.github/workflows/ci.yml`, 4 etapas; corrida verde: https://github.com/felipeganame/CANCHERO-V2/actions/runs/40120787596
Quality gate	Bloquea ante bugs críticos, duplicación > 5 %, cobertura nueva < 80 %	**Cumplido**	`SonarSource/sonarqube-quality-gate-action` en `ci.yml`; bloqueó de verdad el pipeline el 2026-10-24 (78.25 % de cobertura nueva) hasta corregirlo con tests reales, no bajando el umbral
Cobertura sobre código nuevo	80 %	**Cumplido**	SonarCloud, cobertura de código nuevo actual ≥ 80 % (gate en estado *Passed*)
Plan de pruebas y riesgos	Plan versionado, criterios de entrada/salida, riesgos por probabilidad × impacto, trazabilidad requisito → caso → defecto	**Cumplido**	`docs/plan-de-pruebas.md`, `docs/riesgos.md`, `docs/trazabilidad.md`
Defectos	Registrados con pasos, esperado/obtenido, vinculados a su caso de prueba	**Cumplido parcialmente**	`docs/defectos.md` tiene los 4 defectos reales con pasos, causa raíz y el test de regresión que los cubre. No se usó Jira: se usó el propio repositorio (commits + `docs/defectos.md`) como herramienta de registro, trazable vía `git log`
Herramienta nueva	Incorporada con el proceso de 5	**Cumplido**	`docs/herramienta-nueva.md` (k6): requisitos, mercado, prueba de concepto, matriz de

	pasos		decisión, despliegue incremental (gate de 8 VUs en cada push, corrida manual de 20 VUs documentada)

8.2 Nivel de avance (segunda tabla del enunciado)

Aspecto	Exigido (Nivel 3)	Estado	Evidencia
Cobertura mínima sobre código existente	80 %	**Cumplido**	SonarCloud, "Overall Code" → Coverage 82.9 % (2026-10-24)
Smoke test	Ejecutado por el pipeline tras el despliegue	**Cumplido**	Etapa `qa-verification` corre `tests/smoke/smoke.spec.ts` contra QA y, tras el deploy a prod, un curl de salud post-deploy
Integración externa (Supabase)	Flujo E2E automatizado en cada despliegue	**Cumplido**	`tests/e2e/player-flow.spec.ts` y `admin-flow.spec.ts` corren contra QA real en cada push a `main`, antes de habilitar el deploy a producción. Además, `tests/integration-qa/booking-repository.test.ts` prueba RN-01/RN-05 directamente contra el proyecto Supabase de QA real (no Testcontainers) en la misma etapa
Stress y esfuerzo sostenido	Stress ejecutado, con resultados y umbrales	**Cumplido**	Stress (`tests/load/search.js`) corre en cada push con umbrales reales medidos (ver sección 9). Esfuerzo sostenido (`tests/load/soak.js`, 8 VUs, 15 min contra QA real) ejecutado el 2026-10-27: 0.00 % de error, p(95)=551.82 ms, ambos dentro de umbral — ver `docs/metricas.md`

8.3 Metas no cumplidas (honestidad de alcance)

Todas las metas de las tablas 8.1 y 8.2 quedaron cumplidas al cierre de esta entrega. Lo único marcado como "cumplido parcialmente" es el registro de defectos (tabla 8.1): se usó el propio repositorio en vez de Jira, que es una herramienta equivalente pero no la nombrada literalmente en el enunciado.

Fuera de las dos tablas del enunciado, quedan estas limitaciones conocidas, no exigidas explícitamente por la rúbrica pero relevantes:

- **Accesibilidad automatizada (Axe/Lighthouse)** no está integrada al pipeline, afecta la característica de Usabilidad de ISO 25010 (`docs/iso25010.md`).
- **Cross-browser más allá de Chromium** y **limpieza automática de datos de prueba en QA** quedan anotadas como deuda técnica conocida (riesgo R5 en `docs/riesgos.md`).

9. Evolución de las métricas

Métrica	Antes (2026-10-23)	Después (2026-10-24)

Quality gate de SonarCloud	Failed (2 condiciones)	**Passed**
Cobertura sobre código nuevo	22.1 %	**≥ 80 %**
Cobertura total (overall)	21.3 %	**82.9 %**
Bugs de reliability	1 (MAJOR)	0
Tests totales	40	**162** (122 unit/componentes, 32 API, 2 integración Testcontainers, 2 integración QA real, 2 E2E, 2 smoke)
Duplicación de código	0.7 %	0.0 %

El detalle de cómo se llegó a cada número (qué se agregó, qué defecto se encontró en el camino) está en `docs/metricas.md` y `docs/defectos.md` — acá solo se resume la evolución para que coincida con lo que muestra el pipeline y SonarCloud al momento de la entrega, como exige el enunciado.

10. Análisis crítico

Lo que funcionó bien:

- Forzar el quality gate a bloquear de verdad (en vez de dejarlo como paso cosmético) fue lo que expuso la cobertura real del proyecto (22 %) y el bug de reliability — ninguno de los dos se habría corregido si el gate no hubiera frenado el pipeline.
- El patrón de repositorio con doble de prueba (interfaz + implementación real + fake) permitió agregar ~120 tests nuevos sin tocar una base de datos real en la mayoría de los casos, y sin reescribir servicios existentes.
- Probar contra infraestructura real (Supabase, Vercel) encontró 2 defectos (DEF-01, DEF-04) que ningún test contra una base de datos local o un doble de prueba podría haber encontrado, porque dependen de comportamiento específico del proveedor real bajo carga.

Lo que costó más de lo esperado:

- El pipeline de 4 etapas tardó varias iteraciones en estabilizarse (problemas de `pnpm`, builds remotos de Vercel, SSL con Supabase) antes de poder confiar en que un fallo del pipeline era un problema real del código y no de la infraestructura de CI.
- La flakiness de E2E (DEF-02, DEF-04) consumió tiempo de debugging desproporcionado respecto a lo simple de la causa raíz final en cada caso — la lección es que vale la pena sospechar de infraestructura compartida (pool de conexiones, fechas por defecto) antes que del código de la prueba en sí.
- El alcance de autenticación (login real + verificación por email + recuperación de contraseña) se agregó tarde en el desarrollo, lo que obligó a reabrir documentación ya escrita (`docs/riesgos.md`, `docs/plan-de-pruebas.md`) para que siguiera siendo honesta con el estado real del código.

Qué se haría distinto con más tiempo:

- Ejecutar `soak.js` desde el principio del desarrollo en vez de dejarlo para el cierre de la entrega — se terminó corriendo recién al final (2026-10-27), con buen resultado (0 % de error, p95=551ms en

15 min), pero una línea de base más temprana habría dado más margen para reaccionar si el resultado hubiera sido malo.
- Usar branches de Supabase (o una base separada por rama) para QA en vez de reusar siempre la misma, evitando la acumulación de datos de prueba que contribuyó a la flakiness de DEF-04.

11. Conclusiones

Canchero cumple el nivel 3 declarado en los aspectos exigidos por las dos tablas del enunciado: pipeline de 4 etapas con quality gate bloqueante de verdad, cobertura sobre código nuevo y sobre código existente por encima del 80 %, trazabilidad completa requisito → caso → defecto, 4 defectos reales documentados con causa raíz (no fabricados para la entrega), una integración externa (Supabase) con pruebas E2E automatizadas en cada despliegue, una herramienta nueva (k6) incorporada con el proceso de 5 pasos completo, y stress + esfuerzo sostenido ejecutados con resultados y umbrales reales.

Lo único que queda como "cumplido parcialmente" en vez de "cumplido" es el registro de defectos, que se hizo en el propio repositorio en vez de Jira — se prioriza decirlo acá con claridad en vez de reportar un nivel 3 sin matices que no se sostendría ante la pregunta directa en la defensa.

El valor más grande de este proceso no fue cumplir una checklist, sino que el pipeline encontró defectos reales (DEF-01 a DEF-04) durante el desarrollo normal, sin que nadie los buscara a propósito para la entrega — que es exactamente lo que un proceso de calidad bien armado debería producir.